⚡ ARCHITECTURAL SPECIFICATION & TECHNICAL GUIDE

CrowdSourced

Full-Stack Civic Issue Reporting, Geospatial Clustering (DBSCAN), and Real-Time Municipal Triaging Portal

`React 18 + Vite` `Leaflet.js + CartoDB` `Node.js + Express` `Socket.io WebSockets` `PostgreSQL + PostGIS` `Python Flask` `scikit-learn DBSCAN` `Haversine Spatial Metric` `JWT + Cookie Auth`

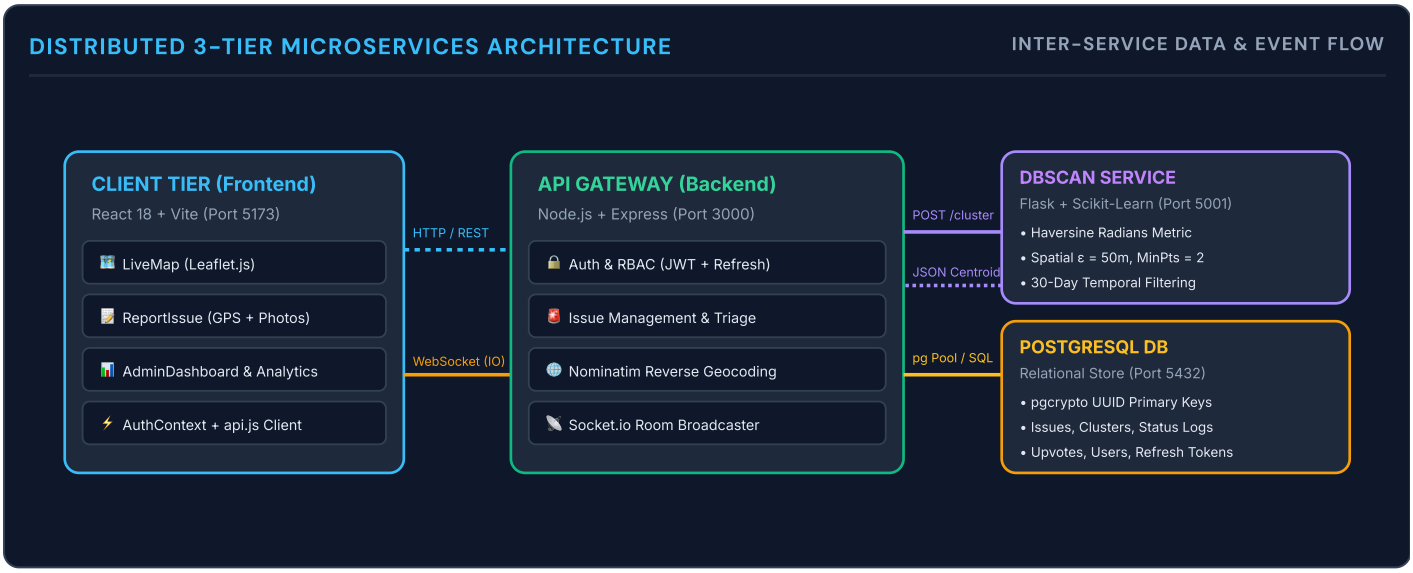
3	37	7	100%
CORE MICROSERVICES	SOURCE & CONFIG FILES	RELATIONAL TABLES	END-TO-END MAPPED

Document Scope: This comprehensive technical document details every single source file, database schema, mathematical clustering algorithm, microservice boundary, and inter-file dependency within the **CrowdSourced** repository. It serves as the definitive reference manual for architects, developers, and municipal stakeholders.

1 System Architecture Blueprint

HIGH-LEVEL OVERVIEW

CrowdSourced is an enterprise civic technology portal designed to streamline municipal grievance redressal. It eliminates duplicate reporting through spatial clustering, automates administrative routing, and provides real-time status visibility to citizens and civic departments.



1. Citizen Reporting

FRONTEND

Citizens submit geotagged civic grievances with real-time GPS pin-dropping, category classification, and image attachment. Instant duplicate checks alert users if an issue was already logged within 50 meters.

2. DBSCAN Clustering

MICROSERVICE

Python microservice clusters incoming reports using Density-Based Spatial Clustering of Applications with Noise (DBSCAN) and great-circle Haversine distance, dynamically calculating centroid coordinates.

3. Municipal Redressal

BACKEND & DB

Department officers triage grouped clusters, assign responsibilities to specialized civic departments (Roads, Sanitation, Electrical), update statuses, and broadcast live WebSocket updates to citizen maps.

2 Master Repository Inventory

COMPLETE CODEBASE MAP

The table below provides an exhaustive index of all files in the `CrowdSourced` project, categorized by architectural layer, runtime responsibility, and language.

File Path	Layer	Tech / Lang	Core Responsibility & Role
`README.md`	ROOT	Markdown	Project overview, architecture summary, and environment quick-start guide.
`backend/src/index.js`	BACKEND	Node.js (ESM)	HTTP & WebSocket gateway server, middleware pipeline, route registration, error handling.
`backend/src/db/pool.js`	BACKEND	Node.js (pg)	PostgreSQL connection pool manager with auto-retry and transactional execution wrapper.
`backend/src/db/schema.sql`	BACKEND	PostgreSQL	DDL definitions for 7 relational tables, foreign key constraints, and 8 performance B-Tree indexes.
`backend/src/db/seed.sql`	BACKEND	SQL	Seed data for Bangalore municipal departments, test accounts, issue clusters, and audit logs.
`backend/src/middleware/auth.js`	BACKEND	Node.js	JWT access token validator, RBAC role-checkers (`requireRole`), and optional auth parser.
`backend/src/middleware/upload.js`	BACKEND	Multer / Node	Memory-buffered multipart photo upload parser and Cloudinary image upload pipeline.
`backend/src/routes/auth.js`	BACKEND	Express Router	Citizen/Dept registration, bcrypt password hashing, login, refresh token rotation, logout.
`backend/src/routes/issues.js`	BACKEND	Express Router	CRUD endpoints for civic issues, cluster querying, citizen upvoting, and status updates.
`backend/src/routes/departments.js`	BACKEND	Express Router	Municipal department listing and department-specific issue/cluster queues.
`backend/src/routes/analytics.js`	BACKEND	Express Router	Aggregated metrics: resolution times, ward breakdowns, duplicate reduction rate calculations.
`backend/src/services/clusterService.js`	BACKEND	Node.js (Axios)	Orchestrates clustering by invoking Python microservice, persists clusters in DB, emits WebSocket events.
`backend/src/services/socketService.js`	BACKEND	Socket.io	Broadcaster for real-time room events (`issue:created` , `status:changed` , `cluster:updated`).
`backend/src/services/nominatimService.js`	BACKEND	Axios / OpenStreetMap	Reverse geocodes (lat, lng) into administrative municipal wards and formatted street addresses.
`clustering/app.py`	CLUSTERING	Python (Flask)	Flask microservice hosting `/health` and `/cluster` REST endpoints with CORS.

File Path	Layer	Tech / Lang	Core Responsibility & Role
`clustering/dbscan.py`	CLUSTERING	scikit-learn / NumPy	DBSCAN geospatial clustering engine using Haversine spherical distance and centroid calculations.
`clustering/requirements.txt`	CLUSTERING	Pip Manifest	Python dependencies: `Flask`, `flask-cors`, `scikit-learn`, `numpy`, `python-dotenv`.
`frontend/src/main.jsx`	FRONTEND	React DOM	React 18 root mounting script with `BrowserRouter` and strict mode wrapper.
`frontend/src/App.jsx`	FRONTEND	React Router	Application layout shell, global navigation bar, route declarations, and `AuthProvider`.
`frontend/src/context/AuthContext.jsx`	FRONTEND	React Context	Global authentication state, JWT token storage, fast demo role-switching (Admin/Dept/Citizen).
`frontend/src/services/api.js`	FRONTEND	Axios Client	HTTP client with JWT interceptor, automated offline fallback engine, and in-memory mock store.
`frontend/src/mockData.js`	FRONTEND	JavaScript	Rich mock dataset for Bangalore civic issues, clusters, departments, and user credentials.
`frontend/src/index.css`	FRONTEND	Vanilla CSS	Design tokens, dark-mode glassmorphism theme, typography, responsive grids, Leaflet styling.
`frontend/src/components/Navbar.jsx`	FRONTEND	React Component	Navigation header, dynamic role badge, one-click role switcher, mobile hamburger menu.
`frontend/src/components/IssueCard.jsx`	FRONTEND	React Component	Interactive issue card with category icon, status badge, upvote button, and cluster count.
`frontend/src/components/IssueDetailModal.jsx`	FRONTEND	React Component	Full issue inspector with status transition controls, department assignment, and audit log.
`frontend/src/components/Toast.jsx`	FRONTEND	React Component	Animated notification toast component for success, warning, and error alerts.
`frontend/src/pages/LiveMap.jsx`	FRONTEND	React (Leaflet)	Interactive dark-mode GIS map with clustered issue markers, filters, and sidebar feed.
`frontend/src/pages/ReportIssue.jsx`	FRONTEND	React Page	Citizen report form with HTML5 GPS geolocation, interactive pin selector, and photo upload.
`frontend/src/pages/AdminDashboard.jsx`	FRONTEND	React Page	Municipal administrative workspace for filtering, triaging, and reassigning civic clusters.
`frontend/src/pages/Analytics.jsx`	FRONTEND	React (Chart.js)	Executive analytics with charts for category frequency, ward distribution,

File Path	Layer	Tech / Lang	Core Responsibility & Role
			and triage SLA.
`frontend/src/pages/MyReports.jsx`	FRONTEND	React Page	Citizen's personal issue tracking dashboard with live status indicators and audit timelines.
`frontend/src/pages/AuthPage.jsx`	FRONTEND	React Page	Unified citizen/department authentication interface with pre-filled demo account buttons.

3 Backend Architecture & Services Deep-Dive

NODE.JS + EXPRESS + POSTGRESQL

The backend is structured as an event-driven RESTful API and WebSocket gateway built with Node.js and ES Modules. It coordinates database transactions, external microservice calls, geocoding, and client synchronization.

backend/src/index.js

SERVER ENTRY

Primary Function: Creates the HTTP server, initializes Socket.io on CORS-enabled origins, binds middleware (CORS, Cookie Parser, JSON 10MB body limits), mounts modular API routers, and defines central error handlers.

Key Code Symbols: `app`, `server`, `io`, `initSocket(io)`, `app.use('/api/issues', ...)`.

backend/src/db/pool.js

DATABASE POOL

Primary Function: Configures `pg.Pool` connection manager targeting PostgreSQL with SSL and idle timeouts. Exports safe helper methods `query()` and `withTransaction()` to ensure ACID compliance during multi-table writes.

Key Code Symbols: `pool`, `query(text, params)`, `withTransaction(callback)`.

Database Schema & Relational Structure (`backend/src/db/schema.sql`)

The schema contains 7 tables designed with UUID primary keys and strict referential integrity.

Table Name	Primary Key	Foreign Keys	Purpose & Constraints
`departments`	`id (UUID)`	None	Municipal agencies (BBMP Roads, BESCOM, BWSSB, Solid Waste Management).
`users`	`id (UUID)`	`department_id → departments.id`	Citizens and municipal officers with bcrypt password hashes and role constraints.
`issue_clusters`	`id (UUID)`	`assigned_department_id → departments.id`	DBSCAN-clustered geographic centroids (lat, lng), category, member count, and aggregate status.
`issues`	`id (UUID)`	`user_id → users.id`, `cluster_id → issue_clusters.id`	Individual civic complaints with GPS coordinates, photos, ward names, and upvote counters.
`status_logs`	`id (UUID)`	`issue_id → issues.id`, `changed_by → users.id`	Immutable audit trail recording all status transitions, officer notes, and timestamps.
`upvotes`	`(user_id, issue_id)`	`user_id → users.id`, `issue_id → issues.id`	Composite primary key preventing duplicate voting on identical complaints.
`refresh_tokens`	`id (UUID)`	`user_id → users.id`	Cryptographic SHA-256 hashed 7-day refresh tokens enabling secure silent session renewal.

> ⚡ **Performance Indexes**
>
> B-Tree indexes are explicitly defined on `issues(category)`, `issues(status)`, `issues(cluster_id)`, `issues(user_id)`, `issues(created_at)`, and `status_logs(issue_id)` to ensure sub-millisecond query performance over millions of civic records.

Backend Services & Route Controllers

services/clusterService.js

SERVICE

Mechanics: Fetches all issues in a category reported within 30 days, POSTs them to the Python clustering service, and wraps results in a database transaction to create or update `issue_clusters` and link member issues. Emits `cluster:updated` over Socket.io.

Exported Functions: `recluster(category)`, `findNearbyCluster(lat, lng, category)`.

services/nominatimService.js

SERVICE

Mechanics: Communicates with OpenStreetMap Nominatim API via HTTP to perform reverse geocoding on GPS coordinates. Extracts administrative suburbs, neighborhoods, or quarters to auto-populate the `ward` column.

Exported Functions: `detectWard(lat, lng)`.

routes/issues.js

ROUTER

Mechanics: Manages issue lifecycle: `POST /` (multipart upload, duplicate check, geocode, async recluster trigger), `GET /` (pagination & filtering), `GET /clusters` (map grouping), `POST /:id/upvote`, `PATCH /:id/status`, and `POST /:id/assign`.

routes/analytics.js

ROUTER

Mechanics: Generates executive reports using PostgreSQL aggregate filters: totals by status, top 10 wards by complaint volume, average resolution time (in hours) calculated from status audit logs, and duplicate reduction rate %.

4 Python Spatial Clustering Microservice

PYTHON + SCIKIT-LEARN + DBSCAN

The clustering microservice (`/clustering`) is an autonomous Python service running on Flask. It solves the critical municipal challenge of **citizen report duplication** by applying mathematical spatial clustering over latitude and longitude coordinates.

clustering/app.py

FLASK API

Role: Hosts the microservice endpoints on port `5001`. Provides `GET /health` for container orchestration checks and `POST /cluster` for executing batch spatial clustering.

Request Body: `{ category, issues: [{id, latitude, longitude, created_at}], eps_meters: 50, min_samples: 2, time_window_days: 30 }`

Response Body: `{ status: "success", cluster_count: N, clusters: [{ centroid_lat, centroid_lng, member_count, issue_ids }] }`

clustering/dbscan.py

ALGORITHM ENGINE

Role: Pure mathematical engine implementing Density-Based Spatial Clustering of Applications with Noise (DBSCAN) using the spherical Haversine metric.

Key Steps:

1. Temporal sliding window filtering (default 30 days).
2. Coordinate conversion from degrees to radians.
3. Epsilon conversion: \(\epsilon_{ ext{rad}} = rac{ ext{eps_meters}}{R_{ ext{earth}}}\) with \(R = 6,371,000 ext{m}\).
4. DBSCAN execution with `metric='haversine'` and `algorithm='ball_tree'`.
5. Centroid calculation by averaging member latitudes and longitudes.

Mathematical & Spatial Mechanics of DBSCAN

Standard Euclidean distance (\(\sqrt{\Delta x^2 + \Delta y^2}\)) creates severe geographic distortion when applied to spherical GPS coordinates. The clustering service converts degrees to radians and applies the **Great-Circle Haversine Formula**:

$$d = 2R \cdot \operatorname{rcsin}\left(\sqrt{\sin^2\left(rac{\Delta \phi}{2} ight) + \cos(\phi_1)\cos(\phi_2)\sin^2\left(rac{\Delta \lambda}{2} ight)} ight)$$

With \(\epsilon = 50 ext{ meters}\) and \(ext{MinSamples} = 2\), two reports within 50 meters of each other coalesce into a single municipal cluster. Single reports (noise / \(ext{label} = -1\)) are returned as distinct 1-member clusters to guarantee zero lost reports.

CLUSTERING EXECUTION & PERSISTENCE LIFECYCLE

END-TO-END ASYNCHRONOUS PIPELINE

STEP 1 **Citizen Submits Issue:** `routes/issues.js` receives geotagged report, persists it to `issues` table, and triggers `recluster(category)` asynchronously (non-blocking HTTP response).

STEP 2 **Data Retrieval:** `clusterService.js` queries all issues in the same category created within the last 30 days via `fetchIssuesForClustering()`.

STEP 3 **Microservice Call:** Node.js POSTs coordinates array to `http://localhost:5001/cluster`. Python Flask passes data into `run_dbscan_clustering()`.

STEP 4 **Atomic DB Persistence:** `clusterService.js` uses `withTransaction()` to upsert `issue_clusters` records and batch-update `issues.cluster_id`.

STEP 5 **Real-Time Broadcast:** Node emits `cluster:updated` via Socket.io to all connected municipal dashboards and citizen map views.

5 Frontend Architecture & UI Component Map

REACT 18 + VITE + LEAFLET

The frontend (`/frontend`) is a modern Single Page Application styled with custom glassmorphism design tokens and dark-mode themes in vanilla CSS. It features an offline-resilient API layer that seamlessly switches to in-memory mock data if the backend is disconnected.

Application Pages & Views

pages/LiveMap.jsx — GIS MAP

Description: Interactive Leaflet map powered by CartoDB dark tiles. Renders cluster markers with dynamic sizing based on member count, category icons, category/status filters, and a synchronized sidebar list.

Key Features: Radius circles for clusters, popup inspectors, fly-to bounds navigation, and instant upvoting.

pages/ReportIssue.jsx — REPORTING FORM

Description: Citizen issue submission workflow. Captures user GPS via browser Geolocation API, allows manual map pin placement, performs auto-reverse-geocoding, and accepts photo uploads.

Duplicate Detection: Checks coordinates against existing clusters and alerts the citizen with a direct upvote option before creating a duplicate.

pages/AdminDashboard.jsx — TRIAGE PORTAL

Description: Operational control center for municipal officers and administrators. Provides tabular and card-based cluster triaging, department assignment dropdowns, status progression actions, and CSV export.

pages/Analytics.jsx — ANALYTICS

Description: Executive metrics dashboard powered by Chart.js. Renders Doughnut charts for category distribution, Bar charts for top ward complaints, resolution SLA timers, and duplicate reduction indicators.

pages/MyReports.jsx — CITIZEN VIEW

Description: Personal tracking feed showing all grievances reported by the logged-in citizen. Displays cluster status badges, municipal department assignments, and timeline audit logs.

pages/AuthPage.jsx — AUTHENTICATION

Description: Tabbed Login and Registration interface with quick-login buttons for demo personas (Super Admin, Municipal Officer, Citizen Rahul).

Core Components & State Management

AuthContext.jsx — STATE

Stores authenticated user profile, access tokens, and exports `useAuth()` hook with role boolean helpers (`isAdmin` , `isDeptOfficer` , `isCitizen`) and instant role switching.

services/api.js — CLIENT API

Unified Axios client with automatic Bearer token injection. Contains a full in-memory fallback engine with Haversine distance calculator and mock data synchronizer for offline demo reliability.

IssueDetailModal.jsx — UI COMPONENT

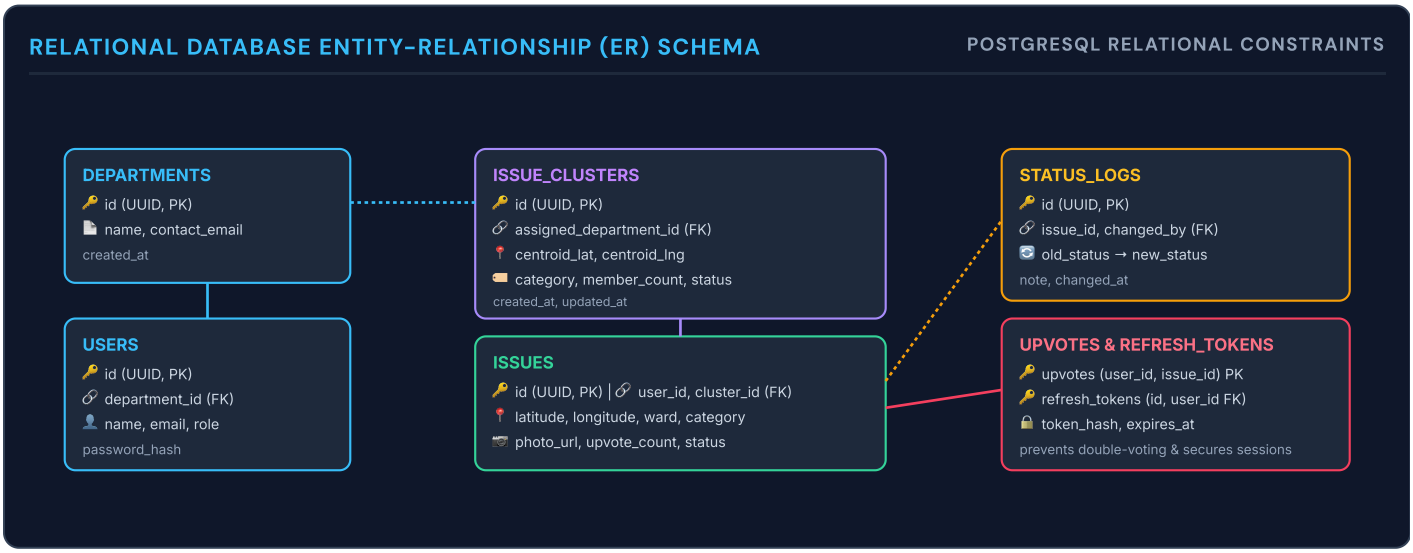
Modal dialog showing photo preview, location map, sibling reports in the same cluster, status transition buttons, notes input, and an immutable audit timeline.

6 Inter-File Dependencies & Relational Model

SYSTEM INTEGRATION MAP

Understanding how files import, invoke, and depend on each other is crucial for maintaining and scaling the platform.

Source File	Direct File Dependencies & Imports	Interaction Mechanism & Purpose
`backend/src/index.js`	`socketService.js`, `routes/auth.js`, `routes/issues.js`, `routes/departments.js`, `routes/analytics.js`	Initializes WebSocket server and mounts REST endpoints at `/api/*`.
`backend/src/routes/issues.js`	`db/pool.js`, `middleware/auth.js`, `middleware/upload.js`, `services/nominatimService.js`, `services/clusterService.js`, `services/socketService.js`	Handles multipart payloads, validates JWTs, reverse geocodes coordinates, executes SQL queries, and triggers clustering.
`backend/src/services/clusterService.js`	`db/pool.js`, `services/socketService.js`, `clustering/app.py` (HTTP)	Pulls recent issues from DB, sends payload to Python Flask microservice, and persists clusters within SQL transactions.
`frontend/src/App.jsx`	`context/AuthContext.jsx`, `components/Navbar.jsx`, `pages/LiveMap.jsx`, `pages/ReportIssue.jsx`, `pages/AdminDashboard.jsx`, `pages/Analytics.jsx`, `pages/MyReports.jsx`, `pages/AuthPage.jsx`	Wraps all routes in `AuthProvider` and renders navigation shell.
`frontend/src/pages/LiveMap.jsx`	`services/api.js`, `components/IssueCard.jsx`, `components/IssueDetailModal.jsx`, `context/AuthContext.jsx`	Fetches clusters and issues from API client, binds Leaflet markers, and opens detail inspector modals.
`frontend/src/services/api.js`	`mockData.js`, `backend/src/routes/*` (HTTP)	Attempts HTTP requests to backend; intercepts failures and routes queries through in-memory mock datasets.

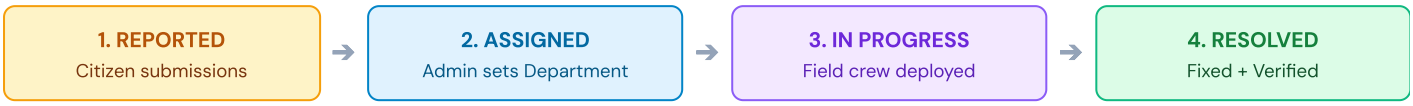


7 Execution Workflows & Environment Setup

OPERATIONAL GUIDE

1. Triaging & Status Resolution Lifecycle

When a civic issue is reported, it follows a strict state transition machine enforced by backend route validation:



2. Authentication & JWT Rotation Workflow

The authentication subsystem uses short-lived JWT Access Tokens (15-minute lifespan) paired with cryptographically secure, SHA-256 hashed Refresh Tokens stored in HTTP-Only, SameSite cookies (7-day lifespan).

Token Issuance (Login)

SECURITY

Upon login, `routes/auth.js` issues a signed JWT containing `sub (user.id)`, `role`, and `department_id`. A random 48-byte hex string is generated, hashed with SHA-256, and stored in `refresh_tokens` while the raw string is placed in a secure cookie.

Silent Token Refresh

SECURITY

When an access token expires, frontend Axios interceptor hits `POST /api/auth/refresh`. The backend verifies the cookie token hash against the database, confirms non-expiry, and returns a fresh 15-minute access token without requiring user re-authentication.

3. Environment Variables & Port Allocations

Service	Port	Variable Name	Default / Description
Frontend	`5173`	`VITE_API_URL`	`http://localhost:3000/api` — Target Node.js gateway API.
Backend	`3000`	`DATABASE_URL`	`postgresql://user:pass@localhost:5432/crowdsourced`
Backend	`3000`	`JWT_SECRET`	Secret key for signing 15-minute JSON Web Tokens.
Backend	`3000`	`CLUSTERING_SERVICE_URL`	`http://localhost:5001` — Target Python DBSCAN Flask service.
Clustering	`5001`	`DBSCAN_EPS_METERS`	`50.0` — Maximum spatial radius in meters to form a cluster.
Clustering	`5001`	`DBSCAN_MIN_SAMPLES`	`2` — Minimum report count required to form a dense cluster.